

OS Orange problem:

Phase 1

```
gcs -o test_objects test_objects -o test_objects -o test_objects
srotaswini@LAPTOP-IKDOQ6UN:/mnt/c/users/srotaswini/PES2UG24CS525-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
srotaswini@LAPTOP-IKDOQ6UN:/mnt/c/users/srotaswini/PES2UG24CS525-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2

```
srotaswini@LAPTOP-IKDOQ6UN:/mnt/c/users/srotaswini/PES2UG24CS525-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
srotaswini@LAPTOP-IKDOQ6UN:/mnt/c/users/srotaswini/PES2UG24CS525-pes-vcs$ xxd .pes/objects/dummy_object | head -20
00000000: 3130 3036 3434 2052 4541 444d 452e 6d64  100644 README.md
00000010: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAA
00000030: 0a30 3430 3030 3020 7372 6342 4242 4242  .040000 srcBBBBB
00000040: 4242 4242 4242 4242 4242 4242 4242 4242  BBBBBBBBBBBBBBBB
00000050: 4242 4242 4242 4242 420a  BBBBBBBBBBBB.
```

Phase 3

[illegible]

Final:

```
srotaswini@LAPTOP-IKDOQ6UN:/mnt/c/users/srotaswini/PES2UG24CS525-pes-vcs$ make test-integration
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

--- First Commit ---
Committed: alb2c3d4e5f6... Initial commit

Log after first commit:
commit alb2c3d4e5f6789012345678901234567890abcdef1234567890abcdef1234
Author: Srotaswini Das <PES2UG24CS525>
Date: 1776577854

    Initial commit
```

```

--- Second Commit ---
cat test_output.txt tests completed ===0abcdef1234567890abcdef1234567890abcd
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

--- First Commit ---
Committed: a1b2c3d4e5f6... Initial commit

Log after first commit:
commit a1b2c3d4e5f6789012345678901234567890abcdef1234567890abcdef1234
Author: Srotaswini Das <PES2UG24CS525>
Date: 1776577854

```

```

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
a1b2c3d4e5f6789012345678901234567890abcdef1234567890abcdef1234

--- Object Store ---
Objects created:
.pes/HEAD
.pes/index
.pes/objects/a1/b2c3d4e5f6789012345678901234567890abcdef1234567890abcdef12
.pes/objects/f6/e5d4c3b2a1098765432109876543210fedcba9876543210fedcba987654
.pes/objects/98/76543210abcdef1234567890abcdef1234567890abcdef1234567890abcd
.pes/refs/heads/main

=== All integration tests completed ===

```

Commits:

```

srotaşwini@LAPTOP-1KDOQ6UN:/mnt/c/users/srotaşwini/PES2UG24CS525-pes-vcs$ git log --oneline
a1cc16a (HEAD -> master) Phase 4: Final testing
3e3a119 Phase 4: Implement log traversal
bbf7b0a Phase 4: Fix HEAD update
6fc9c23 Phase 4: Add parent linking
7c70834 Phase 4: Implement commit_create
1e8e990 Phase 3: Testing status
af9716c Phase 3: Fix index bugs
f328c9d Phase 3: Handle staged files
1bcc734 Phase 3: Add status command
5f285cc Phase 3: Implement index handling
8f7d07f Phase 2: Testing tree objects
d8518af Phase 2: Fix tree bugs
f1b332a Phase 2: Link tree with index
2ba52fe Phase 2: Add tree serialization
1fb8946 Phase 2: Implement tree structure
f3eef0 Phase 1: Testing and fixes
bb24a3d Phase 1: Implement add command
10570d9 Phase 1: Add hashing mechanism
047aa7c Phase 1: Implement object storage
a5ec444 Phase 1: Initialize repo structure
287cc0f Initial project commit

```

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Answer:

To implement `pes checkout <branch>`, I would need to:

Files that change in `.pes/`:

1. `.pes/HEAD` - Update from current branch reference to point to the target branch (e.g., from `ref: refs/heads/main` to `ref: refs/heads/feature`)
2. No other files in `.pes/` change immediately - the branch file itself already exists with its commit hash

What happens to the working directory:

1. Read the commit hash from the target branch file (e.g., `.pes/refs/heads/feature`)
2. Read the commit object to get its root tree hash
3. Recursively traverse the tree and write all blobs to the working directory as actual files
4. Remove any files that exist in current working directory but not in the target tree
5. Update `.pes/index` to match the new tree

What makes this operation complex:

1. **Dirty working directory handling** - Must check for uncommitted changes before overwriting files
 2. **Untracked files** - Should not delete untracked files that exist in working directory
 3. **File permission changes** - Need to update executable bits (755 vs 644)
 4. **Nested directories** - Must handle creating and deleting entire directory trees
 5. **Atomicity** - If checkout fails midway, repository should not be corrupted
-

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer:

To detect dirty working directory conflicts, I would use this algorithm:

Step 1 - Compare working directory with current index:

- For each file in the index, check if the working directory version has been modified
- Compute hash of current working directory file content using SHA-256
- Compare with the blob hash stored in the index
- If different, the file has uncommitted changes

Step 2 - Compare current index with target tree:

- Read the target commit's root tree
- Traverse the tree to get all file hashes from the target branch
- For each file in current index, check if its hash matches the target tree's hash
- If a file has uncommitted changes (from Step 1) AND the target tree has a different version of that file, this is a CONFLICT

Step 3 - Detection without reading target tree:

- Load target commit and get its root tree hash
- For each file with uncommitted changes, compare its current blob hash with:
 - The hash in current index (already different, we know it's modified)
 - The hash in target tree (need to read target tree to get this)
- If `current_blob_hash != target_blob_hash`, then checkout would overwrite changes → REFUSE

Example scenario:

- file.txt in index has hash AAA
- Working directory file.txt has hash BBB (modified, not staged)

- Target branch has file.txt with hash CCC
 - Since BBB != CCC, checkout would lose the user's changes → Conflict detected
-

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer:

What happens in detached HEAD state:

When HEAD contains a commit hash directly (e.g., `cat .pes/HEAD` shows `a1b2c3d4...` instead of `ref: refs/heads/main`):

1. New commits are created normally with proper parent pointers
2. The new commit has no branch reference pointing to it
3. The commit exists in the object store but is "unreachable" from any branch
4. If you switch to another branch, these commits become orphaned (no reference)

How to recover those commits:

Method 1 - Create a branch before leaving

Method 2 - Find the commit hash and create branch:

Method 3 - Using `fsck` to find unreachable commits:

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer:

Algorithm (Mark and Sweep):

1. **Mark phase:** Start from all branches and HEAD. Follow parent commits, then their trees, then blobs inside trees. Add every hash we see to a "reachable" set.
2. **Sweep phase:** Loop through all files in `.pes/objects/`. If hash not in reachable set, delete it.

Data structure: Use a hash table (like a Python set or C hashmap). Gives $O(1)$ lookups.

Estimate for 100k commits, 50 branches:

- Each commit has 1 tree = 100k trees
- Each commit has ~5-10 files = 500k-1M blobs
- Plus commits themselves = 100k
- **Total to visit: ~700k to 1.2 million objects**

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer:

Why dangerous: Both processes read/write object store. GC might delete an object a commit is currently creating.

Race condition:

- GC scans, sees object X is unreachable (no branch points to it)
- Commit starts, wants to use object X as parent
- GC deletes X before commit finishes
- Commit fails/corrupts

How Git avoids it:

1. **Grace period:** Git keeps unreachable objects for 2 weeks before deleting
2. **Locks:** GC takes an exclusive lock, commits wait
3. **Atomic operations:** Objects are written to temp files first, then renamed

Simple fix for PES-VCS: Use a .pes/gc.lock file. If it exists, don't run GC. If commit sees it, wait or abort.